

Introduction to AI and Data Analytics Coursework

Student Number: 2683181

2026/05/05

1 Task 1: German Loan Default Prediction

1.1 Data Description

The German Bank dataset contains 1,000 samples, 16 features (7 numerical, 9 categorical), and a binary target `default` (no/yes), with no missing values or duplicates. The class distribution is imbalanced at 70:30 (no/yes), making raw accuracy unreliable and motivating macro F1-score as the primary metric and recall for `default=yes` as the business-critical criterion [6]. Data was split 70/30 (stratified), with `StandardScaler` and one-hot encoding fitted on the training set only to prevent data leakage.

Key patterns from EDA: customers with a negative checking balance show the highest default rate (49.3%), while those with unknown balance show the lowest (11.7%). Defaulters tend to borrow larger amounts and skew slightly younger.

1.2 Methods

Multi-Layer Perceptron (Deep Learning)

MLP was chosen as the deep-learning model because it can learn nonlinear relationships from the pre-processed feature vector [5]. Two architectures were compared via stratified 5-fold CV; Candidate A was selected and refit on the full training set for 34 epochs (determined by early stopping on an 80/20 subtrain/validation split).

Table 1: MLP architecture comparison.

Candidate	Layers	Dropout	L2	Optimiser	Mean CV F1
A (selected)	64, 32	0.3/0.3	0.001	Adam (lr=0.001)	0.6825
B	128, 64	0.4/0.3	0.001	Adam (lr=0.001)	0.6766

Random Forest (Non-Deep Learning)

Random Forest was chosen as the non-deep-learning model because it is robust on mixed tabular data and provides interpretable feature importance [2]. A 12-combination grid search (5-fold CV) was performed; the best configuration is summarised in Table 2.

Table 2: RF hyperparameter search.

Hyperparameter	Search space	Best value
<code>max_depth</code>	{8, 12, 15}	8
<code>min_samples_split</code>	{5, 10}	5
<code>min_samples_leaf</code>	{2, 4}	4
<code>n_estimators</code>	fixed	100
CV F1 (best)		0.6958

1.3 Results and Analysis

RF achieves higher overall F1 and accuracy, while MLP demonstrates superior recall for `default=yes` (Table ??). Figure 1 confirms that the two models exhibit a clear trade-off: MLP produces fewer

Table 3: Test set performance comparison.

Metric	MLP (Candidate A)	Random Forest
F1 macro	0.6731	0.7166
Recall (yes)	0.6889	0.6556
Precision (yes)	0.5000	0.5784
Balanced Accuracy	0.6968	0.7254
Accuracy	0.7000	0.7533

false negatives (28 vs. 31) but more false positives (62 vs. 43), whereas RF achieves stronger overall discrimination.

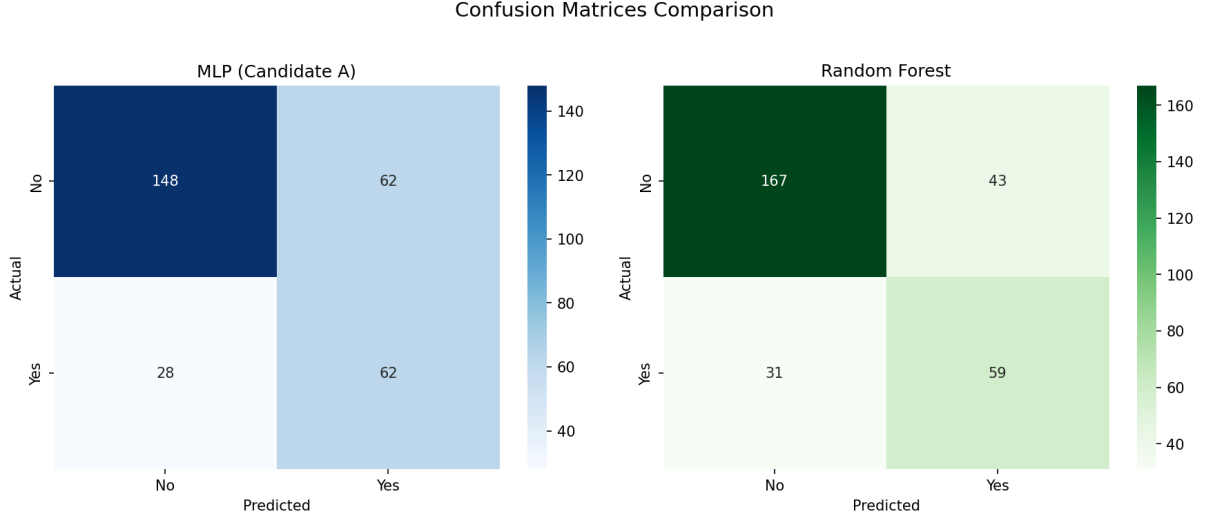


Figure 1: Confusion matrices for MLP (left) and Random Forest (right).

Why results differ. With only 700 training samples, both models face data scarcity. However, this run selected Candidate A (64, 32) over Candidate B, suggesting that the larger architecture does not consistently outperform the smaller one at this dataset size — a sign that MLP capacity is not the binding constraint. RF’s inductive bias towards axis-aligned splits on tabular data provides a more stable advantage, though ensemble predictions remain less transparent than individual trees.

Overfitting. MLP training shows a growing train/validation loss gap from epoch 10, contained by early stopping at epoch 34. RF shows a noticeable train–CV gap, indicating some degree of overfitting, although its CV performance remains stable across the tested depths.

Business cost. In credit risk, missing a defaulter is more costly than a false alarm — reflected in the assumed cost matrix (FN = 3,000 DM, FP = 500 DM). The two models yield near-identical total costs: MLP 115,000 DM vs. RF 114,500 DM, a difference of only 500 DM arising from opposing error profiles. The preferred model therefore depends critically on the true FN:FP cost ratio, which is a business-specific assumption. At a threshold of 0.4, recall rises for both models — especially RF (to 0.8333) — but with reduced macro F1, suggesting threshold selection should be aligned with business priorities rather than fixed at 0.5 by default.

2 Task 2: Financial News Sentiment Classification

2.1 Data Description and Experimental Setup

The dataset contains 5,842 financial news sentences labelled as *negative*, *neutral*, or *positive* by 16 domain-expert annotators [8]. As shown in Figure 2, the dataset is imbalanced: neutral is the largest class (53.6%), followed by positive (31.7%) and negative (14.7%). The sentences are short, mostly between 10 and 30 words, so a maximum token length of 64 covers most samples with little truncation. The class imbalance motivates the use of class-weighted loss and macro-averaged evaluation metrics [6].

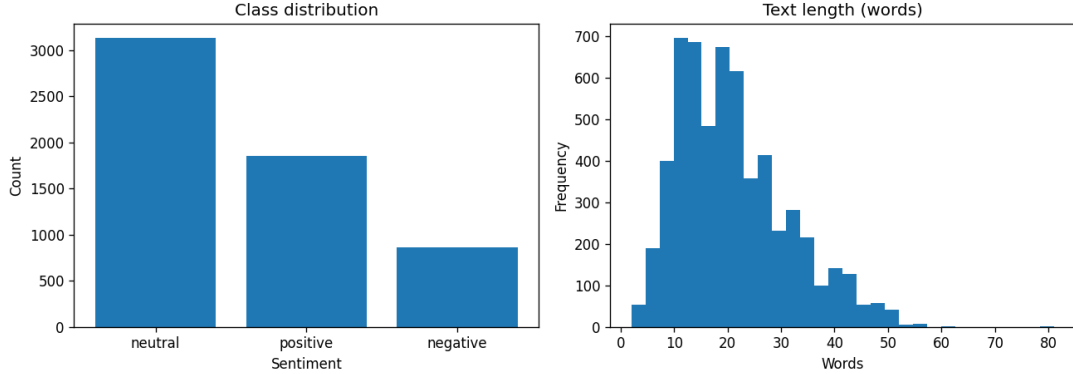


Figure 2: Class distribution (left) and sentence length distribution (right) of the financial sentiment dataset.

The data is split into training, validation, and test sets using a stratified 70/15/15 split, giving 4,089, 876, and 877 samples. The best checkpoint is selected by validation macro F1, and the test set is used only once for final evaluation. Table 4 summarises the main settings.

Table 4: Experimental setup for the DistilBERT classifier

Component	Setting
Base model	<code>distilbert-base-uncased</code>
Data split	Stratified 70/15/15 train/validation/test
Max token length	64
Loss	Class-weighted cross-entropy
Class weights	Negative: 2.26, Neutral: 0.62, Positive: 1.05
Learning rate	2×10^{-5}
Batch size	16
Weight decay	0.01
Max epochs	5
Early stopping patience	2
Model selection	Validation macro F1

2.2 Method and Evaluation Metrics

The method fine-tunes **DistilBERT** (`distilbert-base-uncased`) for three-class financial sentiment classification [10]. DistilBERT is a smaller transformer model distilled from BERT, keeping most of BERT’s language understanding ability while using fewer parameters. Its self-attention layers produce contextualised token representations, which is useful because financial sentiment often depends on context. For example, *loss* in “*net loss contracted*” can indicate improvement rather than deterioration.

Raw sentences are passed directly to the DistilBERT tokenizer and are padded or truncated to 64 tokens. No stopword removal or manual feature engineering is applied, because the pretrained subword tokenizer and transformer layers learn text representations directly from the input. Keeping the raw tokenised sentences is expected to preserve useful financial signals such as company names, numerical values, abbreviations, and directional words such as *from*, *to*, and *not*. These signals could be weakened or removed by stemming or stopword removal. The trade-off is that the model must learn to resolve

financial ambiguity mainly through its pretrained contextual representations. A linear classification head is added on top of the sentence representation, and the full model is fine-tuned end-to-end.

To reduce majority-class bias, training uses class-weighted cross-entropy. This gives more weight to errors on the minority negative class. The expected benefit is higher sensitivity to negative sentiment, while the possible cost is more neutral or positive sentences being predicted as negative. The main strength of this method is its ability to capture contextual meaning with little manual feature design. Its main limitations are higher computational cost and the use of a general-domain model rather than a finance-specific pretrained model.

Four metrics are reported. **Accuracy** gives overall correctness but can be misleading on imbalanced data. **Macro F1** is the main metric because it gives equal weight to all three classes. **Weighted F1** reflects overall performance while accounting for class support, but it can still be dominated by the neutral class. Per-class **precision** and **recall** are used to examine whether the model detects negative samples without over-predicting the negative class.

2.3 Results and Error Analysis

The model was trained for up to 5 epochs with early stopping. The best checkpoint was selected at epoch 3, where validation macro F1 reached 0.739. Final test results are shown in Table 5, and the error pattern is shown in Figure 3.

Table 5: Test set per-class results

Class	Precision	Recall	F1	Support
negative	0.496	0.899	0.639	129
neutral	0.943	0.734	0.825	470
positive	0.827	0.824	0.825	278
macro avg	0.755	0.819	0.763	877
weighted avg	0.840	0.787	0.798	877

A majority-class classifier that always predicts *neutral* would achieve 53.6% accuracy but a macro F1 of only approximately 0.23, since the negative and positive classes would have zero recall. Against this baseline, the DistilBERT model achieves a test accuracy of 0.787 and a macro F1 of 0.763. Neutral and positive both reach F1-scores above 0.82. The negative class has high recall (0.899) but lower precision (0.496), meaning that the model detects most true negative samples but also predicts many neutral or positive sentences as negative. This is consistent with the class-weighted loss, which increases sensitivity to the minority class but raises negative false positives.

There are 187 misclassified test samples. The largest error group is neutral sentences predicted as negative (87 cases), followed by neutral sentences predicted as positive (38 cases) and positive sentences predicted as negative (31 cases). These errors show that the main challenge is financial language whose surface words do not directly match the sentiment label.

Neutral sentences predicted as negative often contain words that look negative but are used factually. For example, “*Finnair estimates to incur a net loss of between EUR2m and EUR2.5m per day*” is labelled neutral because it reports a projection, but the model predicts negative, likely due to words such as *loss*. Positive sentences predicted as negative show a similar issue. In “*net loss contracted to EUR 49m ... from EUR 229m*”, the loss becomes smaller, so the sentence is positive, but the model appears to focus on *loss* rather than the improvement.

The main limitation is that DistilBERT is not specifically pretrained on financial text. A direct improvement would be to use **FinBERT** [1], which may better represent financial expressions. Another improvement is to tune the decision threshold to reduce over-prediction of the negative class. Finally, the dataset contains duplicate sentences; with a row-wise split, some repeated sentences may appear in different partitions and slightly inflate the test score. A stricter future evaluation could use a sentence-level grouped split.

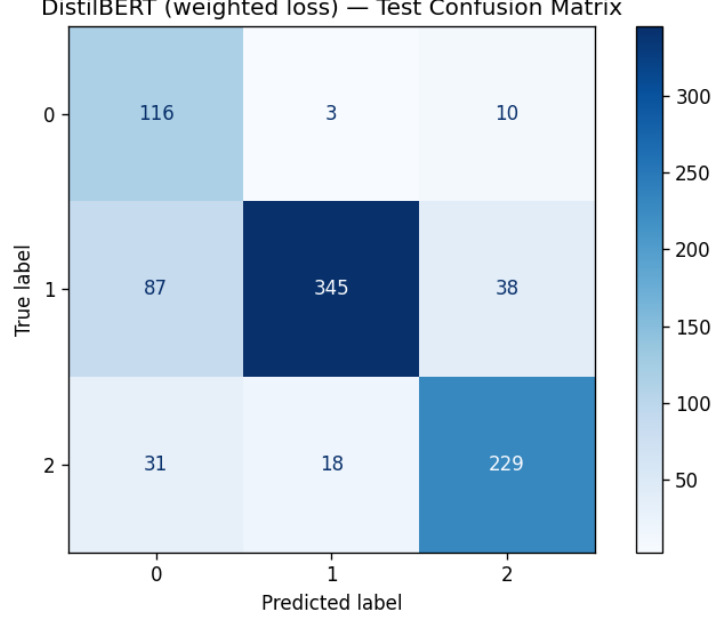


Figure 3: Test confusion matrix for DistilBERT with weighted loss. Labels 0, 1, and 2 correspond to negative, neutral, and positive.

3 Task 3: NanoGPT

3.1 Model Architecture

The implemented model follows the decoder-only transformer architecture of nanoGPT [13]¹, operating at the character level. Each input sequence is represented as an embedding matrix $X \in \mathbb{R}^{T \times d_{\text{embd}}}$. In each attention head, three independent bias-free linear projections produce queries, keys and values:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{embd}} \times d_h},$$

where $d_h = d_{\text{embd}}/n_{\text{head}}$. Scaled dot-product attention is then computed with a causal mask:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + M \right) V.$$

Here, M sets future positions to $-\infty$ before softmax, so each token can only attend to previous tokens and itself during autoregressive prediction.

Multi-head attention is implemented by creating n_{head} independent `Head` modules in an `nn.ModuleList`. Each head learns its own W_Q , W_K and W_V projections. The outputs of all heads are concatenated and projected back to d_{embd} . This implementation is easy to inspect and explain, although it is less computationally optimised than a fused QKV projection.

Transformer layers are stacked using `nn.Sequential`. Each block uses pre-LayerNorm, residual connections and a feed-forward network:

$$X \leftarrow X + \text{MultiHead}(\text{LN}_1(X)), \quad X \leftarrow X + \text{FFN}(\text{LN}_2(X)).$$

The feed-forward network follows a Linear–ReLU–Linear structure and expands the hidden dimension to $4d_{\text{embd}}$ before projecting it back. After the final transformer block, a final LayerNorm and linear head convert contextual representations into logits over the character vocabulary:

$$\text{logits} = \text{LN}_f(X)W_{\text{head}} + b_{\text{head}}, \quad W_{\text{head}} \in \mathbb{R}^{d_{\text{embd}} \times |\mathcal{V}|}.$$

¹Implementation based on <https://github.com/karpathy/nanoGPT>

During training, the logits and targets are reshaped to $(B \cdot T, |\mathcal{V}|)$ and $(B \cdot T)$, and cross-entropy loss is minimised. During inference, only the logits at the final position are used. These logits are divided by a temperature value τ , converted into probabilities by softmax, and sampled autoregressively:

$$p_i = \frac{\exp(\text{logit}_i/\tau)}{\sum_j \exp(\text{logit}_j/\tau)}, \quad \hat{c} \sim \text{Categorical}(p).$$

A lower τ makes the distribution sharper and more repetitive, while a higher τ makes it flatter and more diverse.

3.2 Experimental Setup

The model was trained on the Tiny Shakespeare dataset, a 1.1M-character corpus of Shakespeare’s works. The text was tokenised at the character level, yielding a vocabulary of 65 unique characters. The corpus was split 90/10 into training (1,003,854 tokens) and validation (111,540 tokens) sets. All experiments used a fixed random seed of 42 for reproducibility. The shared base configuration is summarised in Table 6.

Table 6: Base configuration for Task 3 experiments.

Category	Hyperparameter	Value	Category	Hyperparameter	Value
Architecture	<code>n_embd</code>	64	Training	<code>batch_size</code>	32
	<code>n_head</code>	4		<code>max_iters</code>	5,000
	<code>block_size</code>	64		<code>lr</code>	3×10^{-4}
	<code>dropout</code>	0.0		Optimiser	AdamW [7]
Evaluation	<code>eval_interval</code>	500	Evaluation	<code>eval_iters</code>	50

3.3 Hyperparameter Study: Number of Layers

The number of transformer blocks (`n_layer`) was selected as the key hyperparameter, as depth directly determines the model’s capacity to build hierarchical contextual representations. All other settings were fixed to the base configuration to isolate its effect. Table 7 summarises the final results.

Table 7: Final training and validation loss for the two layer configurations.

Model	Params	Train Loss	Val Loss	Gap
Model A (<code>n_layer=2</code>)	112,193	1.8881	1.9659	0.0778
Model B (<code>n_layer=4</code>)	211,777	1.7458	1.8859	0.1401

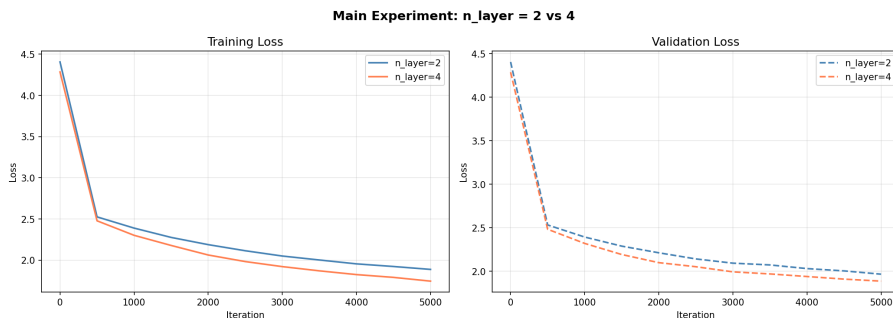


Figure 4: Loss curves for `n_layer=2` and `n_layer=4`.

As shown in Figure 4, Model B achieved consistently lower training and validation losses throughout the entire training process, not merely at the final checkpoint. This suggests that the additional transformer blocks improved learning under this controlled setup, rather than producing only an incidental final-checkpoint difference. However, the improvement in validation loss (0.08) was modest relative to the

near-doubling of parameter count (112K to 212K), indicating diminishing returns at this scale. Model B also exhibited a larger train-validation gap (0.1401 vs 0.0778), reflecting stronger fitting capacity; yet since validation loss continued to decrease through iteration 5,000 without rebounding, there is no evidence of severe overfitting within the given training budget. Both curves were still declining at the final checkpoint, suggesting that further gains may be achievable with a longer training budget, particularly for the deeper model.

3.4 Text Generation Analysis

Text was generated from Model B (`n_layer=4`) using three temperature values to examine the coherence-diversity trade-off. Each sample was generated with the same random seed for a fair qualitative comparison. A representative excerpt from each temperature is shown below.

[T=0.5] AUTINGEL:
That in the you crow sto stand to conter thee a the that

[T=1.0] KIDY LIDGHABET:
Than is chard'd, Stainn you court of in marced

[T=1.5] Kitlo, far duising forger: Gung, StoWanquen'd.
Evere urgaress likeg-Xaquire of kincially;

At $\tau = 0.5$, the output has the most stable line structure and speaker-label format, but it repeats common fragments such as “the”. At $\tau = 1.0$, repetition decreases and punctuation becomes more varied, giving the most balanced Shakespeare-like dialogue structure. At $\tau = 1.5$, diversity increases, but coherence drops sharply, with unstable capitalisation and noise-like invented words. Overall, the model learned surface conventions such as line breaks, capitalised speaker prefixes and punctuation placement, but it did not reliably learn word-level semantics, spelling consistency or cross-line grammar.

3.5 Additional Experiment: Dropout Regularisation

Dropout was investigated as an additional regularisation strategy using Model B (`n_layer=4`) as the base model. All other hyperparameters were fixed, and only the dropout rate was varied between 0.0 and 0.2. Results are summarised in Table 8.

Table 8: Effect of dropout rate on final training and validation loss (`n_layer=4`).

Dropout	Train Loss	Val Loss	Gap
0.0	1.7458	1.8859	0.1401
0.2	1.8991	1.9753	0.0762

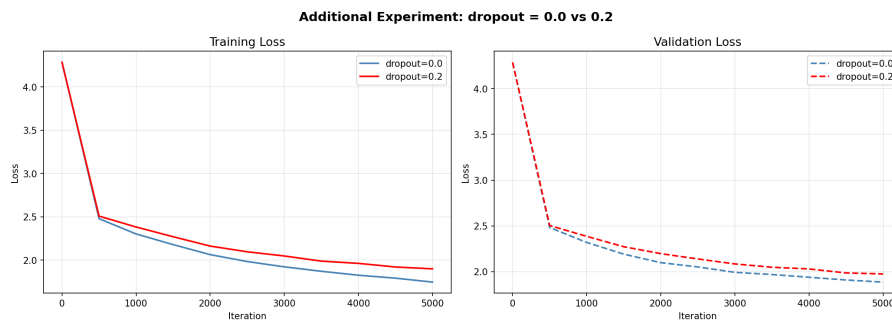


Figure 5: Loss curves for `dropout=0.0` and `dropout=0.2`.

As shown in Figure 5, `dropout=0.2` reduced the train-validation gap from 0.1401 to 0.0762, showing a regularisation effect. However, both training and validation losses were higher throughout training, and final validation loss increased from 1.8859 to 1.9753. This suggests that dropout mainly slowed optimisation rather than improving validation performance in this experiment. Since the no-dropout model did not show severe overfitting, `dropout=0.0` with `n_layer=4` was retained as the best-performing configuration.

4 Task 4: Information Visualisation

4.1 Data Preprocessing

The raw dataset (196 countries, 1980–2024) was reshaped from wide to long format using Python, producing 8,820 country-year observations across four fields: **Country**, **Indicator**, **Year**, and **Inflation**. After type coercion, 868 values (9.8%) were missing, concentrated in earlier decades. To prevent extreme hyperinflation values from collapsing the map’s colour range, a capped field **Inflation_Map_Capped** was created by clipping at the 99th percentile (331.5%); all quantitative analysis retains the original field, and tooltips surface uncapped values to preserve graphical integrity [12]. A country-level summary table (196 rows) was derived to support the COVID impact and long-run views; key fields are listed in Table 9. Venezuela (**Pre_COVID_Avg** $\approx 28,573\%$) was flagged as a hyperinflation outlier, retained in the dataset, and rendered in a dedicated view. Both output files were imported into Tableau and related on the **Country** key.

Table 9: Derived country-level summary fields

Field	Definition
Pre_COVID_Avg	Mean inflation, 2017–2019
COVID_Avg	Mean inflation, 2020–2022
COVID_Impact_Main	COVID_Avg – Pre_COVID_Avg
COVID_Impact_Peak	$\max(2020\text{--}2022) - \text{Pre_COVID_Avg}$
Long_Run_Mean	Mean inflation, 1980–2024

4.2 Visualisation Design and Justification

The workbook contains five worksheets in two dashboards following Shneiderman’s overview-first, details-on-demand principle [11].

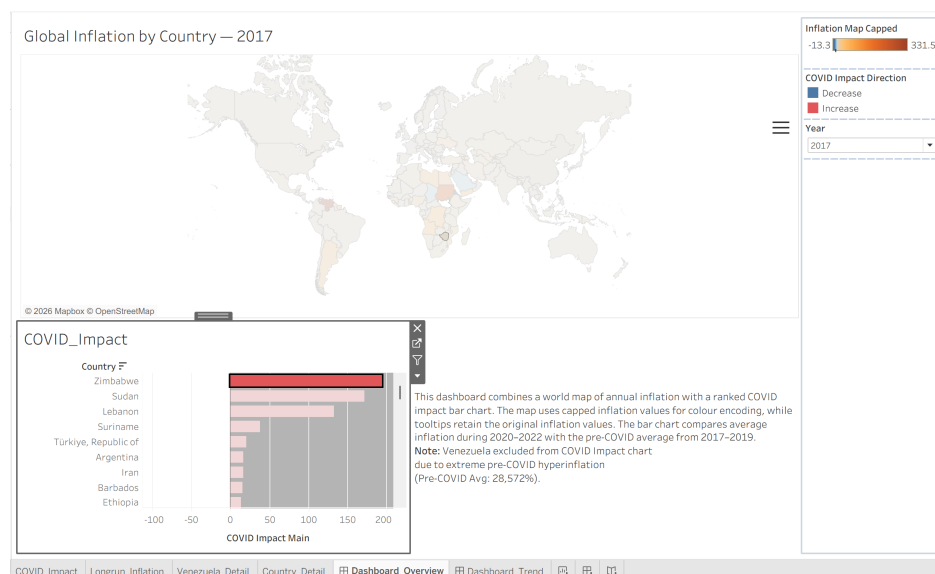


Figure 6: Dashboard.Overview: choropleth world map (top) and COVID impact bar chart (bottom). Map uses capped colour encoding; tooltips retain original values.

Dashboard.Overview (Figure 6) pairs a choropleth world map with a ranked COVID impact bar chart. The map encodes **Inflation_Map_Capped** using a sequential orange scale; a sequential rather than diverging palette is used because inflation has a natural zero reference rather than a symmetric midpoint [3]. A Year dropdown gives explicit temporal control, avoiding the causal misreading that animation can introduce [9]. The COVID bar chart sorts countries by **COVID_Impact_Main** on a common

baseline, the most perceptually accurate encoding for magnitude comparison [4]. Red and blue encode direction of change independently from bar length. Venezuela is excluded via a filter but disclosed in the caption. Selecting a bar highlights the corresponding country on the map; clicking the highlighted shape navigates to `Dashboard_Trend`.

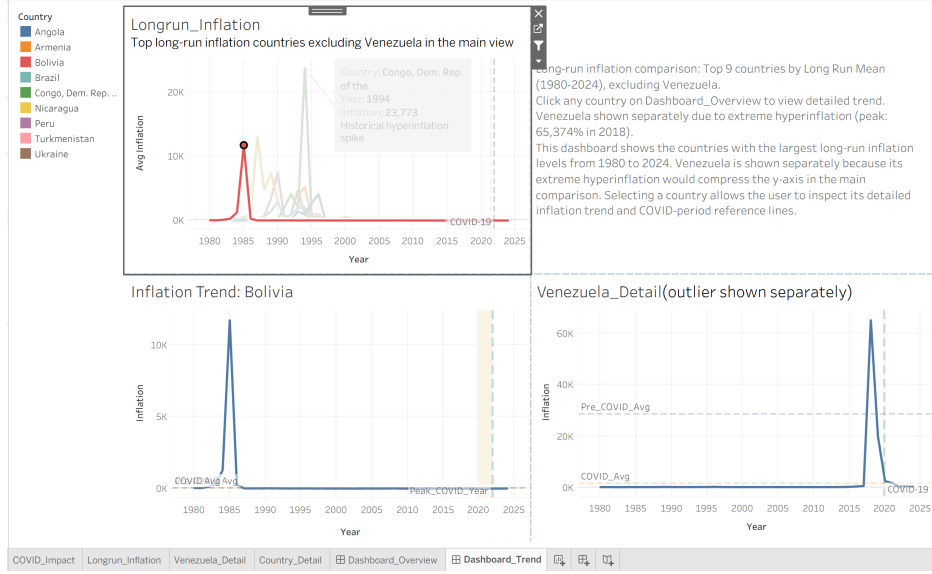


Figure 7: `Dashboard_Trend`: long-run line chart for Top 9 countries (top), country detail with COVID reference lines (bottom-left), and Venezuela outlier on its own scale (bottom-right).

Dashboard_Trend (Figure 7) shows annual inflation from 1980 to 2024 for the Top 9 countries by `Long_Run_Mean`, with Venezuela on a separate fixed panel. Excluding Venezuela from the shared axis is preferable to truncating the y-axis, which would violate graphical integrity [12]. Clicking a country name in the legend highlights its line; clicking the line populates the `Country_Detail` panel via progressive disclosure [11]. The detail view overlays horizontal reference lines for `Pre_COVID_Avg` and `COVID_Avg`, a vertical green dashed line for `Peak_COVID_Year`, and a light-yellow shaded band over 2020–2022, reducing the cognitive effort required to quantify the COVID-period shift [14].

4.3 Findings

COVID-19 Impact on Inflation

After excluding Venezuela and two countries with insufficient data, 193 countries had valid `COVID_Impact_Main` values, of which 165 showed higher average inflation during 2020–2022 relative to 2017–2019. Table 10 summarises the most and least affected economies.

Table 10: Largest COVID-period inflation changes (pp = percentage points)

Country	Pre-COVID Avg	COVID Avg	Impact (pp)
Zimbabwe	88.9%	283.0%	+194.1
Sudan	48.9%	220.4%	+171.5
Lebanon	4.5%	137.0%	+132.5
Suriname	11.1%	48.8%	+37.7
Türkiye	14.2%	34.7%	+20.5
South Sudan	115.2%	17.0%	−98.2
Dem. Rep. Congo	23.2%	9.9%	−13.3
Egypt	19.4%	6.2%	−13.2

Zimbabwe recorded the largest positive impact (+194.1 pp), though its pre-COVID average was already elevated by a domestic monetary crisis (2019 inflation: 255.3%), so the COVID period represents an

intensification of a pre-existing shock rather than an isolated pandemic effect. Lebanon’s surge from a 4.5% baseline was driven primarily by a sovereign debt collapse that began in late 2019. South Sudan’s large negative value (−98.2 pp) reflects reversion from a crisis-driven pre-COVID baseline rather than pandemic-induced deflation. These results are descriptive and should not be interpreted as causal estimates, as the metric does not control for concurrent political or macroeconomic factors.

Countries with Highest Long-Run Inflation

Venezuela leads with a **Long_Run_Mean** of 2,041.7% (2018 peak: 65,374%) and is shown separately. Table 11 lists the next nine countries.

Table 11: Top countries by long-run mean inflation, 1980–2024

Country	Long-Run Mean	Long-Run Max
Congo, Dem. Rep.	791.7%	23,773%
Nicaragua	676.2%	13,110%
Bolivia	311.9%	11,750%
Armenia	291.2%	5,273%
Peru	288.2%	7,482%
Brazil	263.8%	2,948%
Angola	249.8%	4,146%
Ukraine	232.2%	4,735%
Turkmenistan	224.0%	3,102%

The long-run chart shows that extreme inflation was concentrated in short episodes during the 1980s–1990s: Latin American cases reflect the debt crises of that decade, while post-Soviet states experienced acute transitional inflation after 1991. All series converged to low rates by the 2000s, suggesting that structural reforms drove disinflation rather than favourable external conditions. These countries are therefore more accurately described as historically rather than persistently high-inflation economies. A limitation of **Long_Run_Mean** is its sensitivity to missing early-year data, which can overstate rankings for post-Soviet states with no pre-1991 observations.

Global Spatial Patterns

The world map shows that in the 1980s–1990s, high inflation was concentrated in Latin America and sub-Saharan Africa. By 2000, most countries had shifted to low-saturation colours, reflecting broad global disinflation. The 2022 snapshot shows renewed but geographically dispersed intensification across Eastern Europe, the Middle East, and parts of Africa, consistent with post-pandemic supply-chain disruptions and energy price shocks. These regional clustering patterns are not recoverable from tabular rankings alone, demonstrating the interpretive value of the spatial encoding.

References

- [1] Dogu Araci. FinBERT: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Cynthia A. Brewer. Color use guidelines for mapping and visualization. In Alan M. MacEachren and D. R. Fraser Taylor, editors, *Visualization in Modern Cartography*, pages 123–147. Pergamon, 1994.
- [4] William S. Cleveland and Robert McGill. Graphical perception: Theory, experimentation, and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387):531–554, 1984.
- [5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [6] Haibo He and Eduardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [8] Pekka Malo, Ankur Sinha, Pekka Korhonen, Jyrki Wallenius, and Pyry Takala. Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4):782–796, 2014.
- [9] Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014.
- [10] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [11] Ben Shneiderman. The eyes have it: A task by data type taxonomy for information visualizations. In *Proceedings of the IEEE Symposium on Visual Languages*, pages 336–343. IEEE, 1996.
- [12] Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, 1983.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [14] Colin Ware. *Information Visualization: Perception for Design*. Morgan Kaufmann, 2nd edition, 2004.